Identifying Partial Differential Equations with Physics-informed Neural Networks and Sparse Regression

Wongel Nadew, Frank Katjipaha

April 2025

1 Introduction

Partial Differential Equations (PDEs) are indispensable tools in mechanics to understand how systems' state variables vary over a certain domain or evolve through time. They are ubiquitous tools in mechanics to characterize important phenomena, including sound, heat, waves, diffusion, fluid flow, and deformation of materials. Identifying these equations using traditional methods, such as first-principle derivation, is quite a difficult task due to the complex systems becoming arduous; we must resort to new approaches to identify governing equations.

In recent years, Sparse regression has emerged as a powerful tool to discover PDEs from data [4]. However, this method is highly susceptible to inherent data noise, which is exacerbated by derivative calculation. Other methods to handle data noise are insufficient due to the difficulty of balancing the bias-variance tradeoff. To address this challenge, this project leverages Physics-Informed Neural Networks (PINNs) [3] combined with sparse regression to discover the classical diffusion PDE from random walk Monte Carlo simulation data, without prior knowledge of the underlying equation. Previous work specifically on the integration of Physics-informed neural networks and sparse regression was done by Chen et.al [1]. However, their numerical experiments were on synthetic data with Gaussian noise added to numerical solutions. Apart from Physics Informed Neural Networks, there are methods that apply sparse regression using different differentiation methods using the PySINDy package[2].

Key Objectives

- Investigate the application of PINNs for extracting governing PDEs from noisy datasets.
- Implement a hybrid framework integrating sparse regression with PINNs for robust PDE discovery.
- Analyze the impact of hyperparameters on the identified PDEs within model discovery frameworks.

2 Data

We analyzed time-series data from a Metropolis Monte-Carlo simulation acquired from our research lab. The reasons we chose these datasets are:

- **PDE-Agnostic Data Generation** – The simulation is not explicitly informed by any underlying PDE, meaning the data closely resembles real-world scenarios where governing equations are unknown.
- **Controlled Validation**: The ground truth is known– classical diffusion equation with a form $\frac{\partial c}{\partial t} = D\frac{\partial^2 c}{\partial x^2}$. This allows us to rigorously evaluate the performance of the method we are applying and observe the effect of hyperparameters.

To validate the generalizability of this method, we applied it to three distinct datasets, each with different initial and boundary conditions. Data from three different conditions are shown in Figure 1.

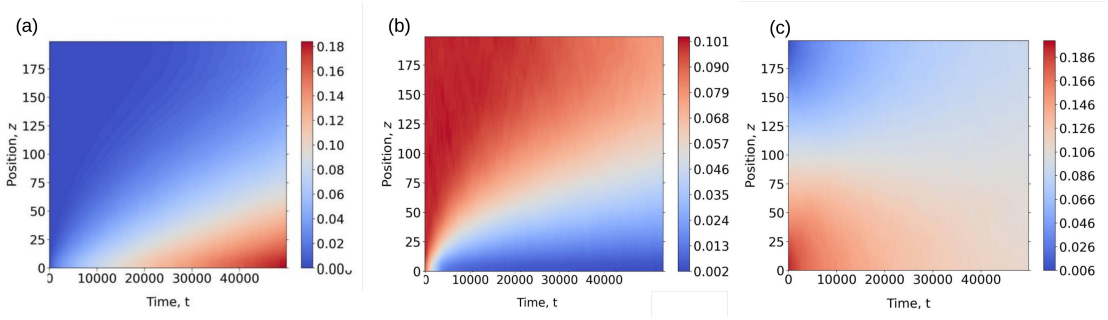


Figure 1: Time-series concentration data from Monte-Carlo simulation, (a) deposition or positive boundary flux condition. (b) deletion or negative boundary flux. (c) Zero boundary flux condition.

2.1 Challenges in handling real-world noise

Unlike synthetic noise, which can be introduced to clean data with quantifiable characteristics, dealing with real-world unknown noise distribution is much more difficult to analyze. This poses a particular challenge when calculating derivatives from noisy datasets, as numerical differentiation amplifies high-frequency noise, making it difficult to balance bias (oversmoothing) and variance (noise sensitivity). Other derivative schemes that can be applied to perform numerical differentiation on datasets with no ground truth include Savitsky-Golay, Kalman filter, and Total Variational Regularization[5].

3 Methods

3.1 Physics Informed Neural Networks

We leveraged Physics-informed neural networks (PINNs) coupled with sparse regression to distill governing equations from spatio-temporal data presented in Section 2. PINNs have gained recognition since Rassi et al[3] demonstrated their effectiveness in solving an inverse problem of retrieving PDEs from data. The problem is a semi-supervised learning in which the grid position of the time series data is used as an input, and the network learns a representation of the data along with coefficients (Λ) of the terms that make up the PDE. We are considering a partial differential equation (PDE) of the form:

$$c_t + \mathcal{N}(c; \Lambda) = 0 \tag{1}$$

where $\mathcal{N}$ is a non-linear operator and Λ is a coefficient vector corresponding to the terms that make up the PDE, and c is the state variable representing particle concentration. To achieve a PDE with only relevant terms, we apply sparse regression.

3.2 Sparse Regression

Sparse regression can identify a parsimonious model from a large candidate function space.

$$\hat{\mathbf{\Lambda}} = \underset{\mathbf{\Xi}}{argmin} \; || \, \mathbf{C_t} - \, \mathbf{\Phi} \, \mathbf{\Lambda} \, ||_2^2 \, + \, \beta ||\mathbf{\Lambda}||_0 \tag{2}$$

where the vector, $\mathbf{C_t}$, represents the time derivative of concentration over the spatio-temporal grid, each row of $\mathbf{\Phi}$ represents all candidate terms calculated at the corresponding temporal-spatial grid point for the flux C_t in that row, and $\mathbf{\Lambda}$ represents the coefficients for the candidate terms in $\mathbf{\Phi}$, which are determined through sparse regression. However, the regularization terms $|\Lambda|_0$ can not be directly incorporated into PINNs since it is non-differentiable. Instead, an L^1 regularization is implemented as a proxy to promote sparsity during the network training.

$$\hat{\mathbf{\Lambda}} = \underset{\mathbf{\Xi}}{argmin} \; || \, \mathbf{C_t} - \, \mathbf{\Phi} \, \mathbf{\Lambda} \, ||_2^2 \, + \, \beta ||\mathbf{\Lambda}||_1 \tag{3}$$

3.3 Combining PINN and Sparse Regression

Sparse regression can be embedded into PINNs by using L^1 regularization. We expect most entries in the Λ vector to be zero, hence a sparse coefficient vector. The architecture of the network used is a simple feed-forward neural network (shown in Figure2) with a custom loss function, which is composed of data loss, physics loss, and regularization loss.

$$\begin{aligned}
\mathcal{L}_{total} &= \mathcal{L}_{data} + \alpha \mathcal{L}_{physics} + \beta \mathcal{L}_{regularization} \\
\mathcal{L}_{data} &= \sum_{i=1}^{N} \frac{||c - \hat{c}||^2}{N} \\
\mathcal{L}_{physics} &= ||C_t - \Phi\Lambda||_2^2 \\
\mathcal{L}_{regularization} &= ||\Lambda||_1
\end{aligned} \tag{4}$$

where c is the actual concentration data collected from the simulation and $\hat{c}$ is the neural network predicted concentration value. The first term in Equation 4 penalizes the error of the neural network representation of the raw data. The last two terms, namely the physics loss and regularization losses, are associated with sparse regression. As an output from the network, we obtain a denoised representation of the concentration data. Using automatic differentiation, we can compute the partial derivatives of the function c. These partial derivatives include c_t, c_z, c_{zz}, and other higher-order derivative terms as desired. Using the partial derivatives computed, the corresponding big data matrix (Φ) is constructed. While the network is learning the function $c(z,t)$, we can also learn the coefficients of the PDE simultaneously. This is achieved by the physics loss. Even though l^1 regularization can achieve sparsity by driving most of the Λ coefficient entries to zero, these entries do not usually become zero. To achieve the sparsity we need, we can extract the big data matrix, Φ, along with the time derivative, C_t, and run sparse regression with L^0 regularization. To run sparse regression, we adopted the algorithm, Sequential threshold ridge regression (STRidge). Once the non-zero terms are identified from sparse regression, we can go back to the PINN and update the active terms of the coefficient and retrain the PINNs until we are satisfied with the consistency of the PDE. To optimize the total loss function, the parameters to be trained are the weights and biases of the network, θ, and the coefficients, Λ.

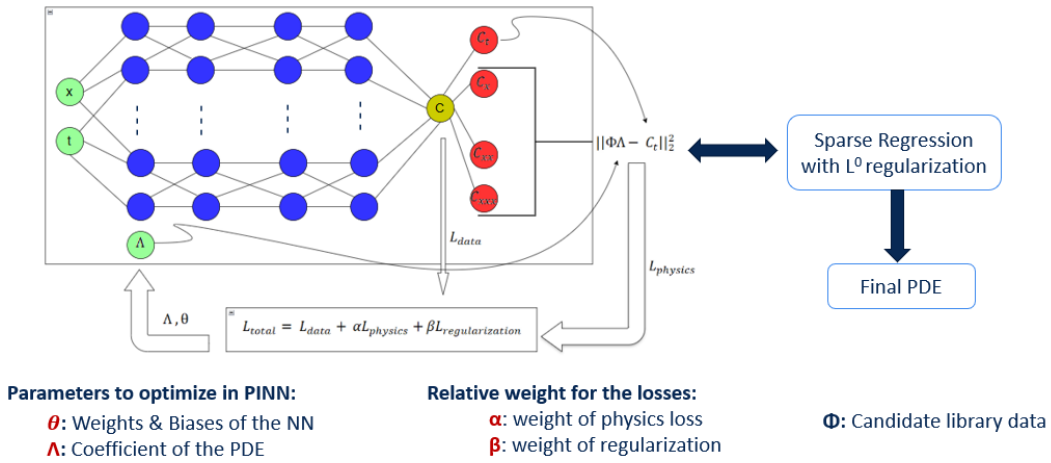


Figure 2: Architecture of PINN with sparse regression

4 Results

Using the PINN-SR framework discussed in the previous section, we train a PINN-SR model on our Monte-Carlo simulation data. We trained the model with Adam optimizer with a learning rate of 10^{-3} for 150 epochs. We used 16 candidate functions, including up to third-order polynomial and a third-order spatial derivative and their exhaustive combination. These terms are:

$$\begin{aligned} &1, \quad c, \quad c^2, \quad c^3, \quad c_z, \quad cc_z, \quad c^2c_z \quad , c^3c_z, \quad c_{zz}, \\ &cc_{zz}, \quad c^2c_{zz}, \quad c^3c_{zz}, \quad c_{zzz}, \quad cc_{zzz}, \quad c^2c_{zzz}, \quad c^3c_{zzz} \end{aligned} \tag{5}$$

Figure shows how various losses in a log scale evolve over epochs. The total loss is low in magnitude

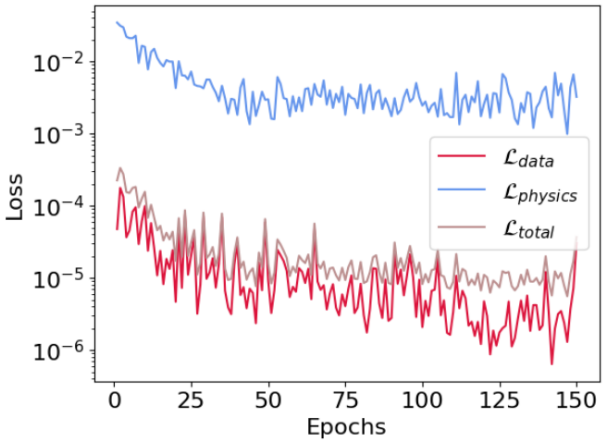


Figure 3: Data loss, physics loss, and total loss versus epochs

compared to the physics loss since the physics loss is rescaled by a small number (10^{-3}). The loss is already minimized at 40 epochs. From the coefficient evolution plot presented in Figure 4. The

size of all the coefficients initially is large. As the network is trained more, the magnitude of most of these coefficients shrinks to zero except for one. The coefficient shown in olive color corresponds to the second-order derivative term, and it persists for longer, indicating that the term is more likely part of the final PDE. Although most of the terms seem to be close to zero, they are not exactly zero. To achieve this, we can extract the data from the PINNs and run sparse regression with l^0 regularization. We run sparse regression with a regularizing parameter $\beta_0 = 0.1$ and retrieved the final form of the PDE. The equation obtained from the PINN-SR model is given as:

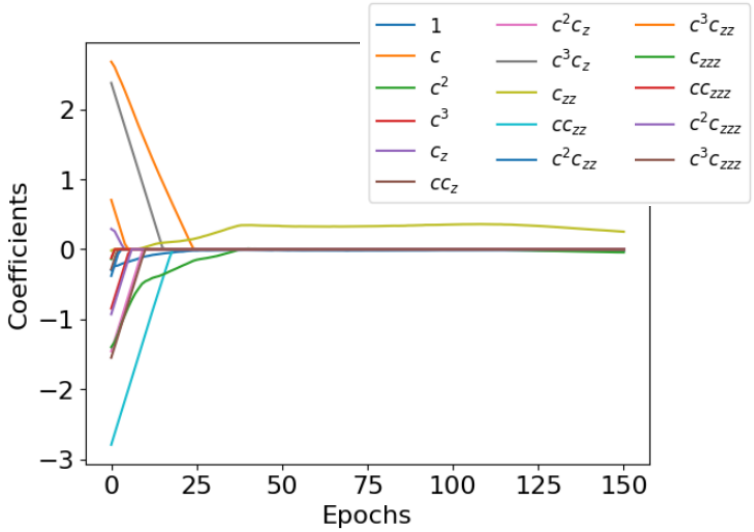


Figure 4: Coefficient evolution against number of epochs elapsed

$$c_t = 0.214 c_{zz} \tag{6}$$

The coefficient in front of the selected term is the diffusivity coefficient of the particles. It has an analytical estimate given by Einstein-Smoluchowski's relation:

$$D_{analytical} = \frac{\langle z \rangle^2}{2nt} = \frac{(1/200)^2}{2 * 3 * 1/500} = 0.20833 \tag{7}$$

Calculating the coefficient error, it is close to 2.88%. Considering the model is trained with noisy data, the coefficient error is acceptable.

5 Discussion

5.1 Hyperparameter selection

One of the major challenging tasks with PINNs is hyperparameter selection. Since the total loss function is composed of three losses, combining them would introduce two loss weight factors. In this case, we have two parameters, α and β, corresponding to relative weights to physics and regularization loss, respectively.

Performing the training with a large α value prioritizes the physics loss while a lower value of α focuses the training on learning the function $c(z,t)$ only. Chen et. al. [1] how to select the magnitude of α, using the ratio standard deviation of the predicted c and the time derivative, c_t ($\alpha_{init} = std(c)/std(c_t)$). However, this approach did not work for the datasets we applied it to. The strategy we found to be effective is using a small value of α initially, and once the network learns a good approximation of our data, $c(z,t)$, we can increase its magnitude to a larger value to impose the physics constraint even more leading to the PDE that most likely generated the data. Initially we used a value of 10^{-5}, and increased it to 10^{-3}.

However, we could not find any straightforward method to tune the beta parameter except for implementing a grid search and observing what gives the best result. Similarly, we applied a grid scan for the regularizing parameter, β_0, in the l^0 regularization. Using various values for β_0 produces numerous PDEs. In this case, we need to generate test data to rule out most of the candidate PDEs and select a single PDE.

5.2 Effection of Regularization parameter β_0

As discussed in the previous subsection, we can do a grid search to search for the best choice for the regularization parameters. To show how the value of β_0 is reflected in the final PDE, we included results with different values on a log scale.

β_0	Equations Obtained
0.01	$c_t = 0.01 - 0.021c - 0.023c_z + 0.499cc_z + 0.212c_{zz} + 0.111cc_{zz} - 0.004c_{zzz} - 0.042cc_{zzz}$
0.1	$0.217c_{zz}$
1	$0.214c_{zz}$

Table 5.2 shows the PDE forms obtained for different β_0 values. For the smallest value, eight terms are picked. However, as the value of β_0 increases, the PDE form becomes sparse. To select among the PDE forms identified, more test datasets are required. Model selection techniques, including Akaike Information Criteria or Bayesian Information Criteria, can be applied to check support for the equations identified.

5.3 Effect of the Data Size

PINNs are known to be less susceptible to data size than sparse regression methods. They sample from sparse points and need far fewer data points than other sparse regression methods. We investigated how much less data they can handle and still yield reasonable results.

Number of data points	Equation Obtained
50,000	$0.217c_{zz}$
20,000	$0.214c_{zz}$
10,000	$0.216c_{zz}$
5000	$0.217c_{zz}$
1000	$0.1111 - 2.045c + 0.684c_z + 5.433cc_z - 0.240c_{zz} + 3.207cc_{zz} - 30.328c^2c_{zz}$
500	$-0.0061 + 0.198c_z + -0.169c_{zzz}$

As shown in Table 5.3, the PINN-SR model can extract the correct PDE with a reasonable coefficient error for a large number of data points to a small number of data points (5000). However, it breaks down for 1000 data points or fewer. This depicts that even though PINN-SR is highly robust for smaller data sizes, it has its limitations in terms of data size.

5.4 PINN-SR with Multiple Datasets

Using multiple datasets for PDE identification has more benefits beyond having more data points to train the model. It also helps to obtain generalized PDEs since these datasets are constructed using different initial and boundary conditions and incorporate various features. Only terms that apply to all datasets are selected, and this promotes the selection of candidate models that generalize well for all types of datasets; hence, the name governing differential equations.

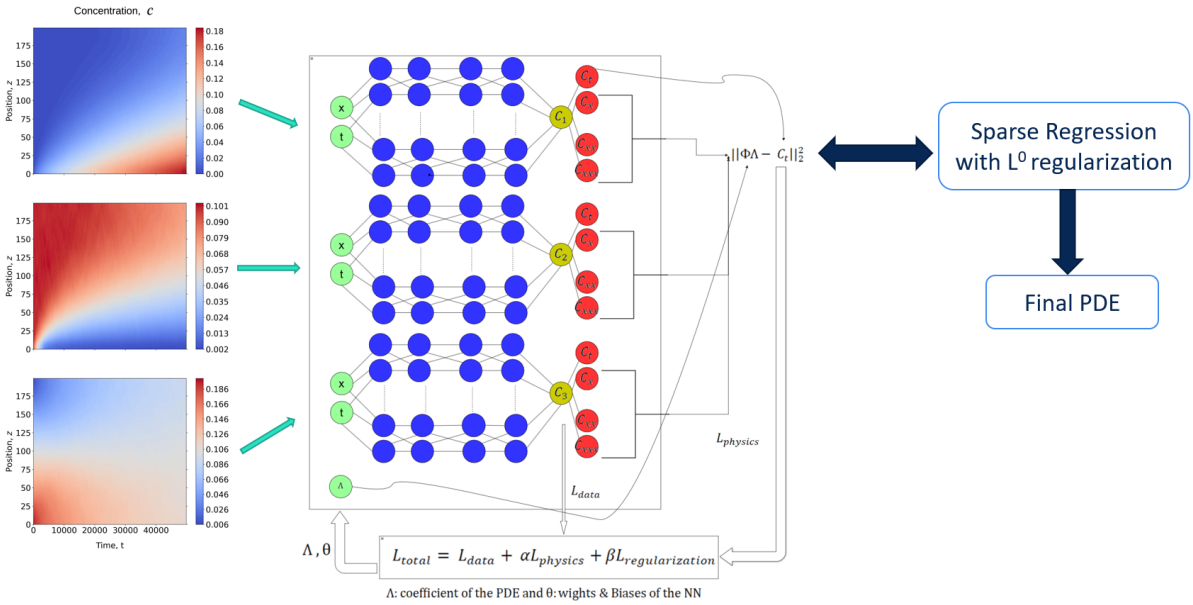


Figure 5: Three datasets used to identify a single governing PDE

5.5 Comparison between PINN-SR and Sparse Regression

5.5.1 Advantages of PINN-SR

PINNs are:

- Robust to noise since neural networks can represent the data distribution, $c(z,t)$, really well.
- Less susceptible to localized noise because they are sampled from a sparse grid of points.
- Known to require less amount of data points compared to other sparse regression methods.

However, they also have some disadvantages, including:

- They could struggle with localized features. Since they sample from sparse grid points, there is a possibility that essential features could be overlooked, and terms corresponding to these features could be neglected.
- Balancing each loss value needs careful hyperparameter optimization, as poor choices might lead to wrong PDEs.

On the other hand, other sparse regression implementations generally have a small number of hyperparameters to tune. Less computation time and easy implementation also make them a good choice. They also offer good control over the model selection process since it is easier to see how changes in hyperparameters affect the final PDE form. However, these methods suffer from issues that arise due to noise. PINN-SR is a better choice to handle noisy datasets than other sparse regression implementations.

6 Conclusion

In this work, we successfully extracted the classical diffusion equation from Monte Carlo simulation data using a Physics-Informed Neural Network (PINN) coupled with sparse regression. The hybrid PINN-SR approach we adopted minimized a composite loss function incorporating data fidelity, physics constraints, and l^1 regularization to learn both the state variable distribution and its underlying PDE structure. Through this framework, we recovered the target PDE with 97% coefficient accuracy, while discovering that progressively enforcing physics constraints during training enhanced model performance. The method's success on this canonical problem demonstrates its potential for governing equation discovery in more complex systems where the underlying dynamics remain unknown. Future work should focus on expanding applications to nonlinear and higher-dimensional systems while conducting systematic studies of hyperparameter sensitivity to improve robustness across diverse physical regimes.

Contributions

Frank Katjipaha: Literature review, model training, code debugging, and writing project report.
Wongel Nadew: Data acquisition, model training, code debugging, and writing project report.

- Used GitHub to communicate changes to the code.
- Held regular meeting to discuss our progress, challenges, and next steps.

Code: Code we are writing to implement the PINN-SR is available on the GitHub page: `https://github.com/Wongel-Yilma/PINN-SR/blob/main/PINN_SR.ipynb`

References

[1] Zhao Chen, Yang Liu, and Hao Sun. Physics-informed learning of governing equations from scarce data. *Nature communications*, 12(1):6136, 2021.

[2] Alan A Kaptanoglu, Brian M de Silva, Urban Fasel, Kadierdan Kaheman, Andy J Goldschmidt, Jared L Callaham, Charles B Delahunt, Zachary G Nicolaou, Kathleen Champion, Jean-Christophe Loiseau, et al. Pysindy: A comprehensive python package for robust sparse system identification. *arXiv preprint arXiv:2111.08481*, 2021.

[3] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

[4] Samuel H Rudy, Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. Data-driven discovery of partial differential equations. *Science advances*, 3(4):e1602614, 2017.

[5] Floris Van Breugel, J. Nathan Kutz, and Bingni W. Brunton. Numerical differentiation of noisy data: A unifying multi-objective optimization framework. *IEEE Access*, 8:196865–196877, 2020.